

Группа 212

Корепин Матвей

Эссе «Что такое Domain-Driven Design?»

Понятие предметной области

Предметная область — это часть реального мира, для которой создаётся программная система: совокупность понятий, правил, процессов и участников, существующих независимо от того, пишется ли для них программа. Например, для интернет-магазина предметной областью является розничная торговля: товары, заказы, оплаты, доставка, скидки, возвраты. Для банковского приложения предметной областью являются счета, платежи, проценты по вкладам, кредитные лимиты. Предметная область осмысленна сама по себе — эксперт в ней, например продавец или бухгалтер, может о ней рассуждать, даже никогда не видев кода программы.

Что такое Domain-Driven Design

Domain-Driven Design, или **предметно-ориентированное проектирование**, — это подход к проектированию программного обеспечения, при котором структура кода строится не вокруг технических удобств вроде баз данных или фреймворков, а вокруг модели предметной области — то есть вокруг понятий и правил, которыми реально оперируют пользователи и эксперты. Термин ввёл Эрик Эванс в книге «Domain-Driven Design: Tackling Complexity in the Heart of Software» в 2003 году. Идея подхода в том, что самая сложная и изменчивая часть программной системы — это не технологии, а бизнес-логика. Если правила предметной области спрятаны внутри функций, работающих напрямую с таблицами базы данных или с примитивными типами вроде чисел и строк, со временем становится всё труднее понимать, что делает программа, а любое изменение требования превращается в риск что-то сломать.

Зачем нужно предметно-ориентированное проектирование

Назначение предметно-ориентированного проектирования — сделать так, чтобы структура кода отражала структуру знаний о предметной области, а не только техническую реализацию. Для этого подход предлагает несколько ключевых идей.

Во-первых, единый язык: разработчики и эксперты предметной области договариваются об общем словаре терминов и используют его одинаково — как в разговоре с заказчиком, так и в именах классов, функций и переменных в коде. Если эксперт называет что-то «заказом», в коде тоже должен быть класс с соответствующим названием, а не абстрактная сущность вроде «записи» или «элемента».

Во-вторых, модель предметной области: код строится вокруг объектов, которые отражают реальные понятия предметной области — их свойства, поведение и связи между ними, — а не вокруг структуры хранения данных. Модель должна воспроизводить не только данные, но и правила: например, правило «нельзя списать со счёта больше, чем на нём есть денег» — это не проверка где-то в интерфейсе, а неотъемлемое поведение самого объекта «Счёт».

В-третьих, ограниченный контекст: одно и то же слово может означать разное в разных частях большой системы. Например, «товар» в контексте склада — это единица учёта с адресом хранения, а «товар» в контексте маркетинга — это карточка с описанием и фотографиями. Предметно-ориентированное проектирование не пытается натянуть одну общую модель на всю систему целиком, а вместо этого явно выделяет границы, внутри которых модель и её словарь остаются согласованными.

В-четвёртых, разделение сложной и простой логики: подход различает содержательную модель предметной области с её поведением и правилами и вспомогательные технические слои — хранение данных, отправку сообщений, отображение интерфейса, —

не позволяя техническим деталям просачиваться внутрь модели и искажать её.

Как знания о предметной области отражаются в структуре программы

На практике это означает, что структура классов и модулей программы должна быть узнаваема экспертом предметной области. Если посмотреть на список классов системы, названия должны напоминать словарь этой области, а не словарь баз данных или веб-фреймворков. Связи между классами тоже должны отражать реальные связи между понятиями: если в реальности заказ состоит из позиций, а каждая позиция ссылается на товар, то и в коде должна быть примерно такая же структура. Более того, правила предметной области, например «сумма заказа не может быть отрицательной» или «скидка не может превышать стоимость товара», должны быть выражены как поведение этих же объектов, а не разбросаны по случайным местам интерфейса или обработчиков запросов.

Собственный пример: система учёта посещений спортивной секции

В качестве собственного примера рассмотрю небольшую предметную область — учёт занятий в спортивной секции, например в кружке борьбы или в бассейне, похожую по духу на пример магазина из конспекта занятия по ООП, но из другой сферы.

Объекты предметной области:

- Спортсмен — человек, посещающий занятия; свойства: имя, дата рождения, группа.
- Тренер — ведущий занятия; свойства: имя, специализация, то есть виды спорта, которые он может вести.
- Группа — набор спортсменов, занимающихся вместе под руководством одного тренера; свойства: название, возрастная категория, расписание.

- Занятие — конкретное проведённое занятие группы в определённый день и время; свойства: дата, длительность, список присутствовавших спортсменов.
- Абонемент — оплаченное право спортсмена посещать занятия в течение периода; свойства: срок действия, количество оплаченных занятий.

Операции над этими объектами тоже взяты из реальной практики секции, а не придуманы под удобство хранения в базе данных: у занятия есть операция отметки присутствия спортсмена; у абонемента есть операция проверки, осталось ли у спортсмена право посетить ещё одно занятие, и операция списания одного занятия после фактического посещения; у группы есть операция назначения нового занятия по расписанию.

Связи между объектами тоже отражают реальную структуру предметной области: спортсмен состоит в одной группе, группу ведёт один тренер, у группы есть много занятий, у занятия — список присутствовавших спортсменов, а у каждого спортсмена — один активный абонемент.

Ключевая мысль предметно-ориентированного проектирования на этом примере в следующем: правило «спортсмен не может быть отмечен присутствующим на занятии, если у него закончился абонемент» должно быть частью поведения самого объекта «Занятие» или «Абонемент», а не отдельной проверкой, спрятанной где-то в коде экрана администратора секции. Тогда это правило будет соблюдаться одинаково независимо от того, откуда оно вызывается — из мобильного приложения, из веб-панели администратора или из будущей интеграции с турникетом на входе, — потому что оно живёт внутри самой модели предметной области, а не дублируется в каждом месте, где эта модель используется.

Связь с материалами занятия по ООП

Domain-Driven Design логично продолжает идеи объектно-ориентированного подхода, разобранные на занятии по ООП: предметная область представляется как совокупность взаимодействующих объектов, у каждого объекта есть состояние и операции, а инкапсуляция скрывает детали реализации за понятным интерфейсом. Разница в акценте: если объектно-ориентированный подход — это набор технических инструментов, таких как классы, наследование и полиморфизм, то предметно-ориентированное проектирование — это дисциплина того, как именно использовать эти инструменты, чтобы модель в коде оставалась верной моделью реального мира, а не превращалась в удобную, но оторванную от смысла структуру данных.